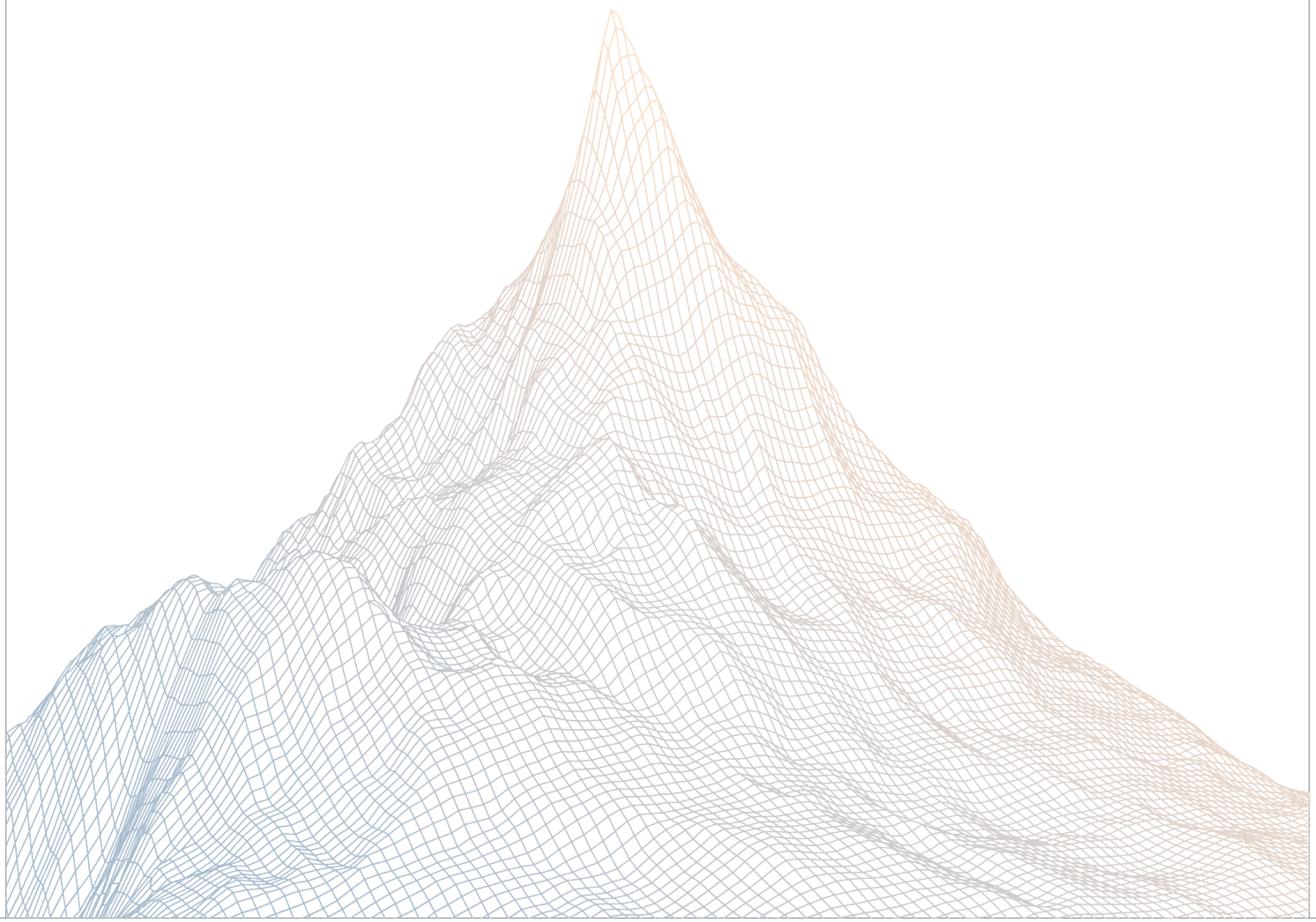
Zenith

Meteora

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: May 15th to May 16th, 2025

AUDITED BY: Peakbolt
Mario Poneder

Contents

1 Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Meteora

Our mission is to build the most secure, sustainable and composable liquidity layer for all of Solana and DeFi.

By using Meteora's DLMM and Dynamic AMM Pools, liquidity providers can earn the best fees and yield on their capital.

This would help transform Solana into the ultimate trading hub for mainstream users in crypto by driving sustainable, long-term liquidity to the platform. Join us at Meteora to shape Solana's future as the go-to destination for all crypto participants.

2.2 Scope

The engagement involved a review of the following targets:

Target	dynamic-bonding-curve
Repository	https://github.com/MeteoraAg/dynamic-bonding-curve
Commit Hash	`df65206a8aee89c2a9f752d6b1e0a5fa75ffa71c`
Files	`Changes in PR-75`

2.3 Audit Timeline

May 15, 2025	Audit start
May 16, 2025	Audit end
June 5, 2025	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	1
Low Risk	2
Informational	1
Total Issues	**4**

3

Findings Summary

ID	Description	Status
M-1	validate_single_swap_instruction() can be bypassed with wrapper program	Resolved
L-1	Snipers can bypass rate limiter fee by splitting over multiple transactions	Acknowledged
L-2	transfer_pool_creator() fails to validate MeteoraDammMigrationMetadata account	Resolved
I-1	Incorrect documentation for get_total_trading_fee()	Resolved

4

Findings

4.1 Medium Risk

A total of 1 medium risk findings were identified.

[M-1] `validate_single_swap_instruction()` can be bypassed with wrapper program

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- ix_swap.rs#L268-L297

Description:

The `validate_single_swap_instruction()` function is designed to stop whale snipers from splitting a large `swap()` into multiple small `swap()` within the same TX to achieve a lower base fee (for rate limiter mode). It does so by checking that there are no duplicate `swap()` in the top-level instructions.

However, this check can be bypassed by using a wrapper program that uses one single instruction to performs multiple `swap()` CPI into the bonding curve program. In that situation, the `validate_single_swap_instruction()` will only see one top-level instruction, which is the single wrapper program's instruction.

That means `validate_single_swap_instruction()` will terminate early at the `current_index == 0` branch and allow the wrapper program to perform multiple `swap()` in the same transaction.

Besides that, the check can also be bypassed even when wrapper program's instruction index is greater than zero. When `current_index > 0`, the for-loop within `validate_single_swap_instruction()` will also terminate early, as the `instruction.program_id` will match the wrapper program and not the curve program.

```
pub fn validate_single_swap_instruction<'c, 'info>(
    remaining_accounts: &'c [AccountInfo<'info>],
) -> Result<()> {
    let instruction_sysvar_account_info = remaining_accounts
        .get(0)
```

```
        .ok_or_else(|| PoolError::FailToValidateSingleSwapInstruction)?;

    // get current index of instruction
    let current_index =
        sysvar::instructions::load_current_index_checked
        (instruction_sysvar_account_info)?;
    if current_index == 0 {
        // skip for first instruction
        return Ok(());
    }
    for i in 0..current_index {
        let instruction =
        sysvar::instructions::load_instruction_at_checked(
            i.into(),
            instruction_sysvar_account_info,
        )?;

        if instruction.program_id == crate::ID
            && instruction.data[..8].eq(SwapInstruction::DISCRIMINATOR)
        {
            msg!("Multiple swaps not allowed");
            return
    Err(PoolError::FailToValidateSingleSwapInstruction.into());
        }
    }

    Ok(())
}
```

Proof-of-Concept:

The following standalone test shows that the `nested_program` will always get index 0 for `load_current_index_checked()` when it is called multiple times via CPI using `wrapper_program`.

- wrapper_program/src/lib.rs

```
use anchor_lang::prelude::*;
use anchor_lang::solana_program::{instruction::Instruction,
    program::invoke};
use anchor_lang::solana_program;

declare_id!("A11111111111111111111111111111111111111111");

#[program]
```

```
pub mod wrapper_program {
    use super::*;

    pub fn call_nested(ctx: Context<CallNested>) -> Result<()> {
        let nested_program_id = ctx.accounts.nested_program.key;


        for i in 0..3 {
            msg!("Calling Nested Program CPI attempt {}", i + 1);

            let ix = Instruction {
                program_id: *nested_program_id,
                accounts: vec![

    AccountMeta::new_readonly(solana_program::sysvar::instructions::ID,
    false),
                ],
                data: an-
    chor_lang::InstructionData::data(&nested_program::instruction::LogIndex
    {}),
            };

            let instruction_sysvar = ctx
                .remaining_accounts
                .iter()
                .find(|acc| acc.key
    == &solana_program::sysvar::instructions::ID)
                .ok_or(ProgramError::NotEnoughAccountKeys)?;

            // Pass sysvar account to CPI
            invoke(&ix, &[instruction_sysvar.clone()])?;

            //invoke(&ix, &[])?; // No accounts required for this CPI
        }

        Ok(())
    }
}

#[derive(Accounts)]
pub struct CallNested<'info> {
    /// CHECK: just passed to CPI
    pub nested_program: UncheckedAccount<'info>,
}
```

- nested_program/src/lib.rs

```rust
use anchor_lang::prelude::*;
use anchor_lang::solana_program::sysvar;

declare_id!("B1111111111111111111111111111111111111111111");

#[program]
pub mod nested_program {
    use super::*;

    pub fn log_index(ctx: Context<LogIndex>) -> Result<()> {
        let instruction_sysvar_account_info =
            ctx.remaining_accounts.get(0).unwrap();

        let idx =
    sysvar::instructions::load_current_index_checked
    (instruction_sysvar_account_info)?;

        require!(idx == 0, ErrorCode::InvalidInstructionIndex);
        msg!("Nested Program: Current instruction index = {}", idx);
        Ok(())
    }
}

#[derive(Accounts)]
pub struct LogIndex {}

#[error_code]
pub enum ErrorCode {
    #[msg("Instruction index is not zero")]
    InvalidInstructionIndex,
}
```

- wrapper_program.ts

```ts
import * as anchor from "@coral-xyz/anchor";
import { Program } from "@coral-xyz/anchor";
import { WrapperProgram } from "../target/types/wrapper_program";
import { assert } from "chai";

describe("anchor-cpi-index-test", () => {
  const provider = anchor.AnchorProvider.env();
  anchor.setProvider(provider);

  const wrapperProgram =
   anchor.workspace.WrapperProgram as Program<WrapperProgram>;
```

```
  const nestedProgramId =
   new anchor.web3.PublicKey("B111111111111111111111111111111111111111");

  it("Calls Nested program via CPI 3 times and logs instruction index",
    async () => {
    const tx = await wrapperProgram.methods
      .callNested()
      .accounts({
        nestedProgram: nestedProgramId,
      })
      .remainingAccounts([
        {
          pubkey: anchor.web3.SYSVAR_INSTRUCTIONS_PUBKEY,
          isWritable: false,
          isSigner: false,
        },
      ])
      .rpc();

    console.log("Transaction signature", tx);
  });
});
```

Recommendations:

This can be resolved by updating `validate_single_swap_instruction()` to check that the top-level `instruction.program_id` always belong to the bonding curve.

Note that this effectively disable CPI into the `swap()` and prevents 3rd party integration.

Meteora: Fixed in PR-79

Zenith: Verified.

4.2 Low Risk

A total of 2 low risk findings were identified.

[L-1] Snipers can bypass rate limiter fee by splitting over multiple transactions

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- fee_rate_limiter.rs

Description:

The rate limiter feature for base fee is designed to penalize whale snipers that buy up the base tokens with large swaps and pump the price. quickly It works by increasing the swap base fee in a step-wise manner when the swap input amount exceeds the `reserved_amount`.

However, it is possible for snipers to split their large swap into multiple smaller `swap()` over multiple transactions, such that each transaction's swap is less than `reserved_amount`.

The snipers could then attempt to frontrun other users by paying priority fees for these swap transactions.

This may not be a problem if the intention of rate limiter feature is to solely penalize large swap, and ensure that whale snipers do not get an unfair advantage over normal users with large swaps. Basically, this levels the playing field, where whale snipers are forced to compete equally against normal users using normal swaps (within `reserved_amount`).

Recommendations:

This issue can be acknowledged as this is an inherent trait of the rate limiter, which distinguishes snipers by swap size.

And the curve creator can avoid this issue simply by choosing the `FeeScheduler` mode, which applies the base fee regardless of swap size.

Meteora: We acknowledge that, integration can mitigate that by apply both rate limiter + dynamic fee.

[L-2] `transfer_pool_creator()` fails to validate `MeteoraDammMigrationMetadata` account

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target

- ix_transfer_pool_creator.rs#L45-L52

Description:

`transfer_pool_creator()` allows the current virtual pool creator to transfer the ownership to a new creator, before the curve is completed or after migration is completed. For the case of migration to DAMM pool, there is an additional check to ensure that the all LP are locked and claimed before pool creator can be transferred.

However, it fails to verify that the `MeteoraDammMigrationMetadata` account belongs to the correct `virtual_pool`, as it is passed in using `ctx.remaining_accounts` without additional validations.

This allows the pool creator to pass in another `MeteoraDammMigrationMetadata` account that has LP locked/claimed to bypass the checks and let the pool creator transfer ownership early.

```
pub fn handle_transfer_pool_creator<'c: 'info, 'info>(
    ctx: Context<'_, '_, 'c, 'info, TransferPoolCreatorCtx>,
) -> Result<()> {
    let mut pool = ctx.accounts.virtual_pool.load_mut()?;

    let migration_progress = pool.get_migration_progress()?;
    let config = ctx.accounts.config.load()?;
    match migration_progress {
        MigrationProgress::PreBondingCurve => {
            // always work
        }
        MigrationProgress::CreatedPool => {
            let migration_option =
    MigrationOption::try_from(config.migration_option)
                .map_err(|_| PoolError::InvalidMigrationOption)?;
            if migration_option == MigrationOption::MeteoraDamm {
                // Can only transfer pool creator after LP claimed + locked
```

```
            //@audit fails to verify MeteoraDammMigrationMetadata belongs
to pool creator's virtual pool
            let migration_metadata_account = ctx
                .remaining_accounts
                .get(0)
                .ok_or_else(|| PoolError::InvalidAccount)?;
            let migration_metadata_loader: AccountLoader<'_,
MeteoraDammMigrationMetadata> =
                AccountLoader::try_from(migration_metadata_account)?;
            let migration_metadata = migration_metadata_loader.load()?;

            require!(
                migration_metadata.partner_locked_lp == 0
                    || migration_metadata.is_partner_lp_locked(),
                PoolError::NotPermitToDoThisAction
            );

            require!(
                migration_metadata.creator_locked_lp == 0
                    || migration_metadata.is_creator_lp_locked(),
                PoolError::NotPermitToDoThisAction
            );

            require!(
                migration_metadata.creator_lp
== 0 || migration_metadata.is_creator_claim_lp(),
                PoolError::NotPermitToDoThisAction
            );
            require!(
                migration_metadata.partner_lp
== 0 || migration_metadata.is_partner_claim_lp(),
                PoolError::NotPermitToDoThisAction
            );
        }
    }
    _ => return Err(PoolError::NotPermitToDoThisAction.into()),
}
```

Recommendations:

This can be resolved by verifying that the `MeteoraDammMigrationMetadata` belongs to the correct virtual pool.

Meteora: Fixed in @9666b94...

Zenith: Verified.

4.3 Informational

A total of 1 informational findings were identified.

[I-1] Incorrect documentation for `get_total_trading_fee()`

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- config.rs#L64

Description:

The comments for `get_total_trading_fee()` states that `MAX_FEE_NUMERATOR = 50%`, which is incorrect as it has been increased to 99% in this release.

```
/// Calculates the total trading fee numerator by combining base fee and
    dynamic fee.
/// The base fee is determined by the fee scheduler mode (linear or
    exponential) and time period.
/// The dynamic fee is based on price volatility and is only applied if
    dynamic fees are enabled.
/// The total fee is capped at MAX_FEE_NUMERATOR (50%) to ensure reasonable
    trading costs.
///
/// Returns the total fee numerator that will be used to calculate actual
    trading fees.
pub fn get_total_trading_fee(
```

Recommendations:

This can be corrected as follows:

```
impl PoolFeesConfig {
    /// Calculates the total trading fee numerator by combining base fee and
    dynamic fee.
```

```
    /// The base fee is determined by the fee scheduler mode (linear or
    exponential) and time period.
    /// The dynamic fee is based on price volatility and is only applied if
    dynamic fees are enabled.
    /// The total fee is capped at MAX_FEE_NUMERATOR (50%) to ensure
        reasonable trading costs.
    /// The total fee is capped at MAX_FEE_NUMERATOR (99%) to ensure
        reasonable trading costs.
    ///
    /// Returns the total fee numerator that will be used to calculate actual
    trading fees.
    pub fn get_total_trading_fee(
```

Meteora: Fixed in @df65206...

Zenith: Verified.